Design Pattern Proxy

Intento e definizione

L'intento del design pattern proxy è fornire un sostituto o surrogato per un altro oggetto (detto target) al fine di controllarne l'accesso. Il client, invece di comunicare direttamente con l'oggetto target, interagisce con il surrogato.

L'obiettivo è rendere questo accesso trasparente: il client non deve sapere se sta parlando con l'oggetto reale o con il proxy, né deve modificare il proprio codice. Per il client infatti, che non si accorge di nulla, cambia solo l'istanza concreta, ma l'interfaccia è identica.

Struttura e partecipanti

Il pattern si basa su quattro componenti principali:

1. **Subject** (soggetto): è un'interfaccia comune sia per il RealSubject che per il Proxy. Grazie a questa interfaccia, il Proxy può essere utilizzato ovunque sia atteso il RealSubject.
2. **RealSubject** (soggetto reale): è l'oggetto vero e proprio che svolge il servizio richiesto e che il Proxy rappresenta.
3. **Proxy**: è l'intermediario che:
   - mantiene un riferimento al RealSubject.
   - implementa la stessa interfaccia del RealSubject.
   - controlla l'accesso al RealSubject, aggiungendo logiche prima o dopo l'inoltro della richiesta.
     - *Ad esempio, il Proxy può non creare subito il RealSubject, ma aspettare finché non serve davvero.*
4. **Client**: l'utilizzatore che invoca le operazioni sull'interfaccia Subject.

Diagramma UML di sequenza

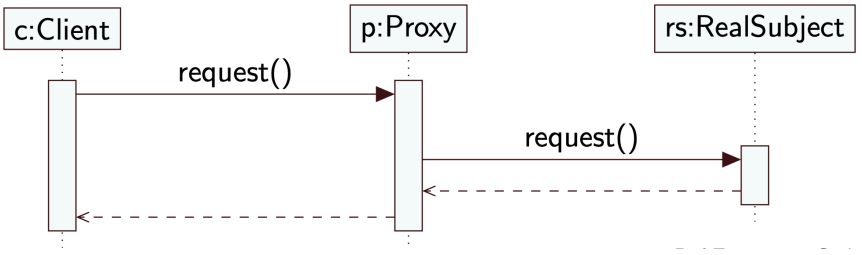
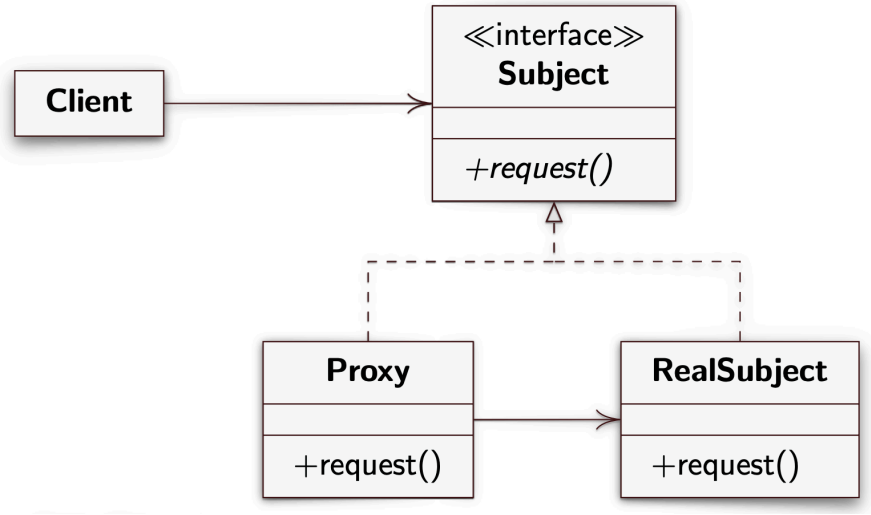


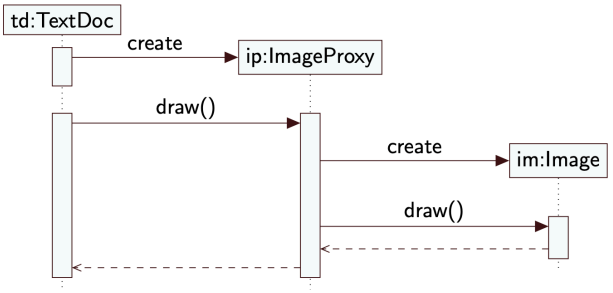
Diagramma UML delle classi



Scenari di utilizzo

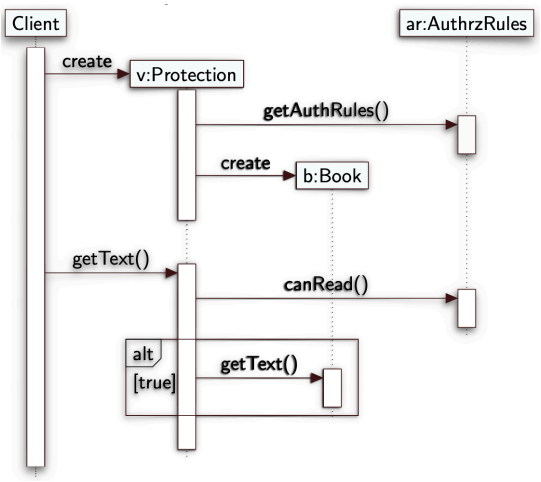
Ottimizzazione (Virtual Proxy):

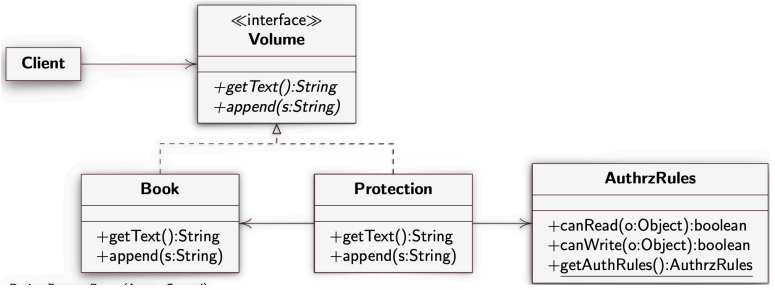
- *Esempio*: all'apertura di un documento con immagini pesanti, si usa un surrogato per le immagini non visibili. L'immagine reale viene caricata in memoria solo quando è effettivamente necessario disegnarla. Quindi in questo caso, il Proxy ImageProxy crea l'istanza di Image solo quando l'editor TextDoc invoca l'operazione draw():



Controllo degli accessi (**Protection Proxy**):

- Si usa quando è necessario proteggere l'accesso a un oggetto in base a permessi o identità. Ad esempio, si vuole poter gestire l'accesso a un libro (Book). Si vuole concedere l'accesso in lettura (getText) o scrittura (append) ai dati di un libro solo se l'utente ha i permessi adeguati. La soluzione in questo caso è:
  - creare un 'interfaccia **Volume** (Subject).
  - la classe **Book** (RealSubject) implementa la logica reale.
  - la classe **Protection** (Proxy) intercetta le chiamate: prima di inoltrare la richiesta a Book, consulta una classe **AuthrzRules** per verificare i permessi.
  - se il permesso è negato, il Proxy blocca la richiesta e non chiama l'oggetto reale.





Remote Proxy (accesso remoto):

- Serve a semplificare l'accesso ad un oggetto che risiede su un altro spazio di indirizzamento (o host), nascondendo la complessità della comunicazione remota.

Copy-on-Write Proxy

- Si tratta dell'ottimizzazione copy-on-write tramite Proxy. La copia di un oggetto costoso viene ritardata finché non avviene una modifica. Se più client leggono l'oggetto senza modificarlo, il Proxy permette loro di condividere la stessa istanza del RealSubject. Appena un client invoca un metodo di modifica (update()), il Proxy crea una nuova istanza esclusiva per quel client.

Conseguenze

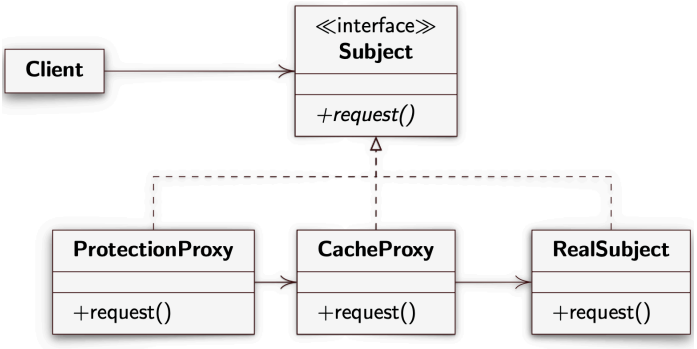
L'utilizzo del Proxy porta diversi vantaggi e considerazioni:

- Introduce un livello di **indirezione** utile per disaccoppiare il client dalla posizione o dallo stato dell'oggetto reale.
- Permette di ottimizzare la creazione di oggetti onerosi (attraverso il caricamento su richiesta).
- Consente di implementare politiche di sicurezza e autorizzazione separate dalla logica di business (comportamento funzionale dell'oggetto reale).
- **Svantaggio**: richiede di implementare una classe Proxy per ogni classe che si vuole proteggere o controllare, portando a una proliferazione di classi nel sistema

Proxy multipli

I Proxy multipli rappresentano una configurazione strutturale in cui è possibile concatenare più oggetti Proxy diversi per gestire l'accesso allo stesso RealSubject.

- **Concatenazione**: invece di un singolo intermediario, si crea una concatenazione di proxy. Un Proxy non chiama direttamente l'oggetto reale, ma passa la richiesta ad un altro Proxy.
- **Interfaccia comune**: tutti i Proxy nella concatenazione e il RealSubject implementano la stessa interfaccia (Subject), permettendo al client e ai proxy successivi di trattarli in modo uniforme.
- *Esempio*:



  - **Client**: invia una richiesta request().
  - **ProtectionProxy**: è il primo a riceverla (gestisce la sicurezza/autorizzazione). Se l'accesso è consentito, inoltra la richiesta al proxy successivo.
  - **CacheProxy**: riceve la richiesta dal ProtectionProxy. Verifica se il risultato è già disponibile in memoria (per ottimizzare le prestazioni). Se non lo è, inoltra la richiesta all'oggetto finale.
  - **RealSubject**: esegue l'operazione reale se nessun proxy precedente ha interrotto la concatenazione o soddisfatto la richiesta.

Il **vantaggio** di questa architettura permette di aggiungere molteplici funzionalità (sicurezza, caching, log, ecc.) in modo modulare, senza dover creare un'unica classe Proxy complessa che gestisce tutto contemporaneamente.

Lo **svantaggio** è che aumenta il livello di indirezione e la complessità del flusso di esecuzione, rendendo più difficile comprendere, mantenere e configurare correttamente il sistema.